Guiding Enumerative Program Synthesis with Large Language Models

Yixuan Li[1(✉)], Julian Parsert[1,2,3], and Elizabeth Polgreen[1(✉)]

[1] University of Edinburgh, Edinburgh, UK
{yixuan.li.cs,elizabeth.polgreen}@ed.ac.uk, julian.parsert@gmail.com
[2] University of Oxford, Oxford, UK
[3] University of Innsbruck, Innsbruck, Austria

Abstract. Pre-trained Large Language Models (LLMs) are beginning to dominate the discourse around automatic code generation with natural language specifications. In contrast, the best-performing synthesizers in the domain of formal synthesis with precise logical specifications are still based on enumerative algorithms. In this paper, we evaluate the abilities of LLMs to solve formal synthesis benchmarks by carefully crafting a library of prompts for the domain. When one-shot synthesis fails, we propose a novel enumerative synthesis algorithm, which integrates calls to an LLM into a weighted probabilistic search. This allows the synthesizer to provide the LLM with information about the progress of the enumerator, and the LLM to provide the enumerator with syntactic guidance in an iterative loop. We evaluate our techniques on benchmarks from the Syntax-Guided Synthesis (SyGuS) competition. We find that GPT-3.5 as a stand-alone tool for formal synthesis is easily outperformed by state-of-the-art formal synthesis algorithms, but our approach integrating the LLM into an enumerative synthesis algorithm shows significant performance gains over both the LLM and the enumerative synthesizer alone and the winning SyGuS competition tool.

1 Introduction

Program synthesis is the task of automatically generating programs that satisfy a given specification. It has applications in planning [13], program analysis [16], data-wrangling [17] and more. The dominant techniques for formal program synthesis are based around enumeration [4,21,37], and a key challenge is how to guide this enumeration to search a huge space of possible programs efficiently. Syntax-Guided Synthesis(SyGuS) [2] allows the user to restrict the space of possible programs using a context-free grammar, and, in later work, this has been extended using pre-trained probabilistic models such as higher-order grammars [27] and neural networks [31], trained on a dataset of solved synthesis problems. However, obtaining these datasets for pre-training is challenging.

In parallel, the use of pre-trained large language models (LLMs) to generate code is rapidly gaining traction, with impressive results being obtained

A. Gurfinkel and V. Ganesh (Eds.): CAV 2024, LNCS 14682, pp. 280–301, 2024.
https://doi.org/10.1007/978-3-031-65630-9_15

on benchmarks with natural language specifications and input-output examples [14]. These benchmarks are very different in style to the logical specifications that formal program synthesis tackles, as most are procedural code, in Python, and solve classic programming exercise questions that might be asked of students or interview candidates, and that one may find in abundance on sources used in training data such as StackOverflow and GitHub. In contrast, formal program synthesis benchmarks, such as those in the SyGuS competition, require functional code, which must satisfy precise logical specifications derived from problems such as program analysis [16], and are certainly less abundant in sources of publicly available code for training machine learning models.

In this paper, we set out to investigate whether off-the-shelf large language models can solve formal program synthesis problems. We craft a library of prompts, which enables us to solve roughly 50% of the SyGuS competition benchmarks. We hypothesize that, in the cases where the LLM returns only incorrect solutions, the correct solutions are most often in the vicinity of the incorrect solutions, and that, by searching in the neighborhood of the incorrect solutions, we may be able to guide an enumerative synthesizer to find a solution faster. To that end, we construct a probabilistic Context-Free Grammar (pCFG) based on the incorrect solutions proposed by the LLM, and use this to guide an enumerative synthesizer within a CounterExample Guided Inductive Synthesis (CEGIS) loop.

Our final contribution is a full integration of these techniques in a novel CEGIS algorithm with an inline syntactic oracle, in the form of an LLM that is queried by an enumerative synthesis phase. We incorporate information obtained during the synthesis search into the queries, prompting the LLM with partially enumerated functions, incorrect solutions, and counterexamples, and requesting that it provide "helper functions", which we use to update the pCFG guiding the enumerator.

We implement all three techniques described above and evaluate them on benchmarks from the Syntax-Guided Synthesis competition. We compare with two baselines: the first is an enumerative synthesizer where all rules in the grammar are given equal likelihood, and the second is cvc5 [7], the state-of-the-art SyGuS solver. All techniques easily outperform the baseline enumerator, and the final technique outperforms cvc5. Our results demonstrate that, whilst large language models do have the potential to make significant contributions in the domain of formal program synthesis, this can currently only be achieved by combining these techniques with existing algorithms in the literature. Enumerative synthesis is not yet obsolete!

The main contributions of our work are as follows: A set of prompts for prompting a pre-trained Large Language Model to solve formal program synthesis problems (Sect. 4.1); A method for guiding an enumerative synthesizer using LLM-generated probabilistic context-free grammars (Sect. 5.1); A novel approach to integrating an LLM into an enumerative synthesizer (Sect. 6); And, finally, an implementation and evaluation of all of the above on benchmark problems taken

from the Syntax-Guided Synthesis competition. The results outperform cvc5, the state-of-the-art synthesizer, as well as our baseline enumerators.

2 Background

Program synthesis focuses on automated program creation that satisfies a high-level specification, which can be comprehensive, such as a basic, unrefined program, or incomplete, like a logical formula or a set of test cases.

Definition 1 (Context-Free Grammar, CFG). *A context-free grammar is a 4-tuple $G = (V, \Sigma, R, S)$. V is a finite set of variables also known as non-terminal symbols. Σ with $\Sigma \cap V = \emptyset$ is called the set of terminal symbols or alphabet. $R \subseteq V \times (V \cup \Sigma)^*$ is a finite relation describing the production rules of the grammar. We define $R_\Sigma = R \cap V \times \Sigma^*$, i.e. the set of rules restricted to those whose right-hand side only consists of terminal symbols. Elements of $(V \cup \Sigma)^*$ are known as words in sentential form. $S \in V$ is the start symbol of the grammar G.*

Given a context-free grammar $G = (V, \Sigma, R, S)$ with $x, y \in (V \cup \Sigma)^*$ and $(\alpha, \beta) \in R$ we say that $x\alpha y$ yields $x\beta y$, written $x\alpha y \rightarrow x\beta y$. We say that x derives y written $x \rightarrow^* y$ if either $x = y$ or $x \rightarrow x_1 \rightarrow \ldots x_n \rightarrow y$ for $n \geq 0$. Finally, we define the *language* of a grammar $\mathcal{L}^G = \{s \in \Sigma^* \mid S \rightarrow^* s\}$. We now introduce two extensions of context-free grammars:

Definition 2 (Weighted Context-Free Grammar, wCFG). *A weighted context-free grammar (wCFG) [29,30] is a 5-tuple $W_G = (V, \Sigma, R, S, W)$ such that (V, Σ, R, S) is a context-free grammar and W is a function assigning a numeric value to each rule $r \in R$.*

Definition 3 (Probabilistic Context-Free Grammar, pCFG). *A probabilistic context-free grammar [29,30] is a 5-tuple $P_G = (V, \Sigma, R, S, \mathbb{P})$ such that (V, Σ, R, S) is a context-free grammar and $\mathbb{P}$ is a probability mass function assigning a probability $\mathbb{P}[r]$ to each rule $r \in R$. $\mathbb{P}_\Sigma$ is the probability mass function that assigns a probability to $\mathbb{P}_\Sigma[r]$ to each rule $r \in R_\Sigma$. A pCFG is a specific instance of a wCFG.*

In general, program synthesis is concerned with the generation (i.e., synthesis) of a program that satisfies a certain specification. Syntax-guided synthesis (SyGuS) describes a standardized function synthesis format that precisely defines a synthesis problem within first-order theories [8]. We will use the notation $\phi[F \mapsto f]$ to denote the replacing of all occurrences of F in ϕ with f while substituting all arguments to f by the arguments of F in the same order.

Definition 4 (Syntax-Guided Synthesis, SyGuS). *A SyGuS problem is a 4-tuple $\langle T, G, \phi, F\rangle$ such that T is a first-order theory, G is a context-free grammar, ϕ is a first-order formula, and F is a function symbol that may occur in ϕ. A solution to a SyGuS problem $\langle T, G, \phi, F\rangle$ is either a function f such that $T \models \phi[F \mapsto f]$ and $f \in \mathcal{L}^G$, or proof that no such function can exist.*

SyGuS closely follows the syntax and semantics of SMT, and hence T usually refers to theories that are also common in SMT. Usually, SMT solvers are queried in the background of SyGuS solvers to verify solution candidates. This connection is made explicit in *Counter-Example Guided Inductive Synthesis* (CEGIS) [39]. CEGIS is a family of algorithms that alternate between a synthesis phase, which searches for a candidate solution that works for a subset of inputs, and a verification phase, where the candidate is checked against all possible inputs. If the verification fails, a counterexample is passed back to the synthesis phase and appended to the subset of inputs used to guide the search. The synthesis phase is often implemented as an enumerative search. An example SyGuS problem is shown in Example 1.

Generative Large Language Models. Generative Large Language Models (LLMs) are advanced Artificial Intelligence (AI) systems based on transformer models and trained on vast datasets to produce human-like text, followed by human-provided instruction prompts [10]. One application of LLMs is generating code from natural language specifications [14].

3 Overview

In this work, we first present a carefully tailored set of prompts that we use to evaluate an LLM's ability to solve formal synthesis problems. We construct an iterative loop where we prompt the LLM, verify the candidate solution, and if the solution fails, we prompt the LLM again.

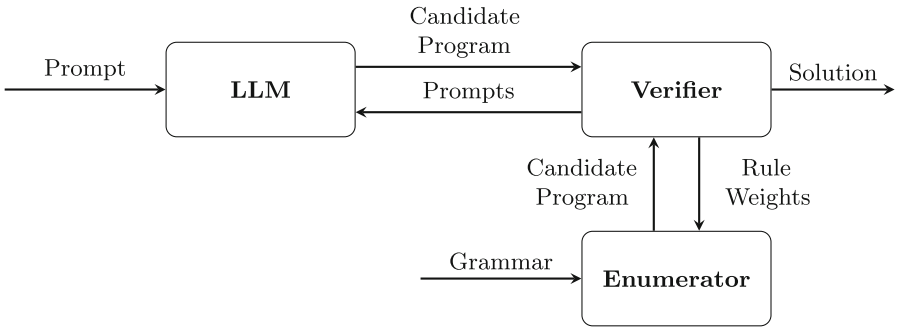

Fig. 1. An overview of pCFG-synth. Both the verifier and the LLM have access to the specification ϕ (which is used to generate the prompt for the LLM, as well as to check whether candidate programs are correct).

We then present two methods for integrating syntactic guidance from pre-trained LLMs into an enumerative CEGIS algorithm. The first method, shown in Fig. 1, prompts an LLM for solutions to the benchmark, and generates a pCFG from these solutions before deploying an enumerative synthesizer, increasing the

chance of the LLM solving the synthesis problem outright. We refer to this method as pCFG-synth. The second method, shown in Fig. 2, integrates the prompting within the enumerative synthesizer, allowing the prompts to incorporate additional information obtained during the synthesis process. Here, instead of asking the LLM to provide a full solution, we ask it to provide helper functions to help "a student" complete the partially enumerated program. We use the responses to augment the set of production rules in the grammar and update the weights across the existing production rules. We refer to this approach, which integrates an LLM into an enumerative synthesizer, as iLLM-synth. In this section, we give an overview of these two approaches. The details of the components of both approaches and their relative performances are found in the subsequent sections. We integrate both approaches with a probabilistic top-down enumerator and a weighted search based on the A^* algorithm [19,27].

```
(set-logic LIA)
(synth-fun fn ((vr0 Int) (vr1 Int) (vr2 Int)) Int)
(constraint (>= (fn vr0 vr1 vr2) vr0))
(constraint (>= (fn vr0 vr1 vr2) vr1))
(constraint (>= (fn vr0 vr1 vr2) vr2))
(constraint (or (= vr0 (fn vr0 vr1 vr2)) (or (= vr1 (fn vr0 vr1 vr2))
    (= vr2 (fn vr0 vr1 vr2)))))
(check-synth)
```

Example 1. A SyGuS specification that asks for a program that synthesizes the maximum of 3 inputs. We omit some the grammar and variable declarations for brevity.

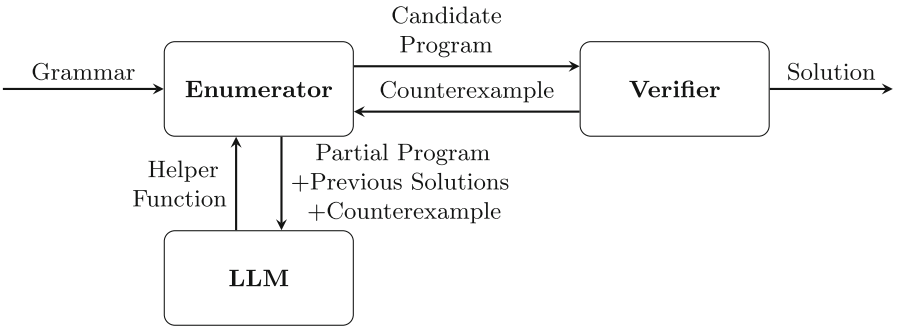

Fig. 2. An overview of iLLM-synth. Both the verifier and the enumerator have access to the specification ϕ (which is used to generate the prompt for the LLM, as well as to check whether candidate programs are correct)

4 Stand-Alone LLM

In this section, we describe how we prompt the LLM as a stand-alone synthesizer. These prompting techniques are then also deployed by pCFG-synth. We use GPT-3.5-turbo as the LLM. Note that the model is not fine-tuned to

this problem setting. Furthermore, we rename any functions and variables in the SyGuS benchmarks to generic names to avoid the LLM producing solutions solely based on the function names.

4.1 Prompting the LLM

We design a library of prompts for program synthesis problems with logical specifications and a single target function to synthesize. These prompts are deployed in an iterative loop, until a correct solution is obtained, or the library of prompts is exhausted.

Prompting is an art rather than a science, but we hypothesize that it is better to ask the LLM to give a solution in a language that is more common in the training data, and then request it to translate it into our desired SMT-LIB, and experiment with both Python and Lisp. On a subset of 50 benchmarks, we observed that soliciting responses in Lisp resulted in a 6% enhancement in the resolution of benchmarks compared to utilizing Python (and both were better than asking for SMT-lib directly). It is also reported in the literature that the efficacy of LLMs can be augmented by including emotional stimuli within prompts [28]. Incorporating the emotional prompt leads to an 8% increase in the likelihood of generating accurate solutions compared to standard, non-emotional prompts in our study. Details of our initial prompting experiments are in the extended version of this paper[1].

The following is an example prompt and response sequence for the LIA example shown in Example 1:

```
(set-logic LIA)
(declare-var vr0 Int)
(declare-var vr1 Int)
(declare-var vr2 Int)
(constraint (>= (fn vr0 vr1 vr2) vr0))
(constraint (>= (fn vr0 vr1 vr2) vr1))
(constraint (>= (fn vr0 vr1 vr2) vr2))
(constraint (or (= vr0 (fn vr0 vr1 vr2)) (or (= vr1 (fn vr0 vr1 vr2))
    (= vr2 (fn vr0 vr1 vr2)))))
(check-synth)
You are a good synthesizer. Do you know what "(define-fun fn ((vr0 Int
    ) (vr1 Int) (vr2 Int)) Int" is doing?
Write only one Lisp-like method "defun fn" without any built-in
    methods or arrays.
Requirements:
1. No built-in functions.
2. Never violate the SMT-LIB constraints above.
3. Pay attention to the define functions.
4. Ensure the response contains one and only one function.
5. Do not include any iterations, BitVec, or Int notations in the
    function body.
Write it correctly, or I will lose my job and 100 grandmothers will
    die. Don't disappoint me.
Write only one Lisp-like method "defun fn" that never violates the SMT
    -LIB constraints above.
```

Prompt 1. Prompt for Example 1.

[1] https://arxiv.org/html/2403.03997.

```
(defun fn (vr0 vr1 vr2)
    (ite (>= vr0 vr1) (ite (>= vr0 vr2) vr0 vr2) (ite (>= vr1 vr2) vr1
         vr2)))
```

Response 1. Response from LLM after Prompt 1.

```
You are close to the right answer. Take another guess. You have to try
     something different, think harder. Write a different Lisp method
     that never violates the SMT-LIB constraints above again.
```

Prompt 2. Prompt requesting a revised solution.

Prompts for Invariant Synthesis. Invariant synthesis is a specific instance of program synthesis: given a pre-condition $pre(x)$, transition-relation $trans(x, x')$ and post-condition $post(x)$, the synthesizer is required to provide an invariant inv that satisfies the following constraint: $\forall x, x'.pre(x) \implies inv(x) \wedge (inv(x) \wedge trans(x, x)) \implies inv(x') \wedge inv(x) \implies post(x)$. We find that LLMs struggle to reason about constraints presented in the above format. Inspired by "chain-of-thought" [42] prompting, where the LLM is asked to provide a step-by-step explanation, we augment our prompting strategy for invariants by asking the LLM first to explain the constraints. After requesting this explanation, we follow the same interactive prompt strategy as before.

Lisp to SMT-LIB Converter. The final prompts in our prompt library are to ask the LLM to convert any functions given in Lisp to correct SMT-LIB functions:

```
You are a good programming language converter. Convert the Lisp
     function to SMT-LIB:
Based on the Lisp code provided above, convert the 'defun' Lisp-like
     code to a corresponding SMT-LIB function. Use SMT-LIB syntax
     starting with (define-fun
Follow these guidelines:
1. Only give me the function definition starting with '(define-fun'.
2. Pay attention to types. If there are bit-vector terms, they need to
     be of the same width.
3. Ensure the SMT-LIB function contains one and only one function
     definition starting with '(define-fun'.
4. Do not include any iterations, BitVec, or Int notations in the
     function body.
5. Use the assigned values from the Lisp code during translation.
6. Do not introduce any variables that do not exist in the Lisp
     function.
Rules for SMT-LIB: +, -, *, ite, >, =, <, >=, <=, and, or, not, true,
     false.
```

Prompt 3. Request for converting Lisp to SMT-LIB code for response 1.

Upon receiving a response from the LLM, we extracted the Lisp program and subjected it to format verification. The resulting SMT-LIB code is represented:

```
(define-fun fn ((vr0 Int) (vr1 Int) (vr2 Int)) Int
    (ite (>= vr0 vr1) (ite (>= vr0 vr2) vr0 vr2) (ite (>= vr1 vr2) vr1
         vr2)))
```

Program 1. LLM-Generated program for Example 1.

5 Synthesis with pCFG Guidance: pCFG-synth

We hypothesize that, if the LLM did not propose a correct solution, the correct solution is likely to be roughly in the same "area" as the incorrect solutions it suggested, and so our synthesis algorithm aims to prioritize this area when searching for candidate programs. For simplicity, we use a simple weighted Context-Free Grammar to represent the area of solutions proposed by the LLM. We then present methods for searching the space: the first is a probabilistic top-down search, shown in Algorithm 3; the second is based on an adaptation of the A^* algorithm [19,27], and we integrate both into CEGIS searches as shown in Algorithm 1. The verification phase in Algorithm 1 is implemented via a call to an SMT solver, which checks, for a candidate solution f, whether there exists an input such that the specification is violated, i.e., $\exists x.\neg\phi[F \mapsto f]$.

Algorithm 1. CEGIS with weighted search

```
1: procedure CEGIS(W_G, φ)
2:     cex ← ∅
3:     while true do
4:         prog ← ENUMERATE(W_G, φ, cex, )
5:         if VERIFY(prog, φ) then
6:             return prog
7:         else
8:             c ← VERIFY.GET_CEX
9:             cex ← cex ∪ {c}
```

5.1 Inferring a Weighted CFG

In this section, we describe how we infer a weighted Context-Free Grammar from the incorrect solutions produced by the large language model.

Definition 5 (Derivations). *Given a context-free grammar G, and a sentence s, the sentence is in the language of the grammar if $S \rightarrow^* s$, where S is the start symbol of the grammar. The derivation of s from S is a sequence of rules such that $S \xrightarrow{r_0} s_1 \xrightarrow{r_1} \dots s_n \xrightarrow{r_n} s$ and $r_0 \dots r_n \in R$. We denote the derivation of s by the sequence of rules $r_0, \dots r_n$ as $D_s = \{r_0, \dots r_n\}$. The left-most derivation is a derivation such that all rules expand the left-most non-terminal symbol in the sentential form.*

From here on in, all derivations are assumed to be the left-most derivation, and we assume the grammar is unambiguous, i.e., there exists a single left-most derivation for any sentence in the language.

Given a set of possible programs $prog \in \mathcal{L}^G$ generated by the language model, we calculate a weight for each rule $r_i \in R$ as the number of times that

rule appears in the left-most derivations of the programs. That is,

$$w[r_i] = \sum_{prog_i \in prog} |r_i| \in D_{prog_i}, \tag{1}$$

where $|r_i|$ is the number of times r_i appears in the derivation. For example, consider Response 1: the weights are calculated as $w[r_1] = 3, w[r_2] = 3, w[r_3] = 3, w[r_4] = 4, w[r_5] = 3$. These correspond to the rules from Example 1:

$r_1 : \texttt{Start} \rightarrow \texttt{(ite StartBool Start Start)}$
$r_2 : \texttt{Start} \rightarrow \texttt{vr0}$
$r_3 : \texttt{Start} \rightarrow \texttt{vr1}$
$r_4 : \texttt{Start} \rightarrow \texttt{vr2}$
$r_5 : \texttt{StartBool} \rightarrow \texttt{(>= Start Start)}.$

Probabilistic Context-Free Grammar. Given a wCFG, we derive a simple pCFG by assuming that the probability associated with a rule $r_i \colon \alpha \rightarrow \beta$ is equal to the weight $w[\alpha \rightarrow \beta]$ of r_i, divided by $|\pi[\alpha]| = |\alpha \times (\Sigma \cup V)^* \in R|$, i.e., the total number of rules that could be applied to α. That is $\mathbb{P}[\alpha \rightarrow \beta] = \frac{w[\alpha \rightarrow \beta]}{|\pi[\alpha]|}$. By extension, $\mathbb{P}_\Sigma[\alpha \rightarrow \beta] = \frac{w[\alpha \rightarrow \beta]}{|\pi[\alpha]}|$ iff $\beta \in \Sigma$ and 0 otherwise.

5.2 Probabilistic Guided Search

The aim of our algorithm is thus to search the area of programs closest to those with the highest weights in the wCFG, or highest probabilities in the corresponding pCFG. We adapt and implement two search methods for doing this: the first is a probabilistic top-down search. To this end, we first introduce the notion of a grammar tree.

Definition 6 (Grammar tree). *We represent the search space as a grammar tree. Given a context-free grammar $G = (V, \Sigma, R, S)$, the graph of sentential forms, or grammar tree, $\mathcal{T}(G)$ defined inductively: S is the root of the tree, and for all $x, y \in (V \cup \Sigma)^*$ with $x \rightarrow y$ and x being a node of the tree, then y is a child node of x.*

To implement our probabilistic guided search, we extend this definition to a probabilistic grammar tree. Given a pCFG, $P_G = (V, \Sigma, R, S, \mathbb{P})$, a probabilistic grammar tree $\mathcal{T}(P_G)$ is a directed labelled graph as defined before, but each edge has a corresponding weight ω given by $\mathbb{P}$. We limit the edges to only those needed for the left-most derivations, and so $\mathcal{E}$ and ω are defined as follows:

$$\mathcal{E} = \{x\alpha y \xrightarrow{\alpha \rightarrow \beta} x\beta y \,|\, \alpha \rightarrow \beta \in R, x \in \Sigma^*, \alpha \in V, \beta, y \in (V \cup \Sigma)^*\},$$
$$\omega[\alpha \rightarrow \beta] = \mathbb{P}[\alpha \rightarrow \beta].$$

Note that this guarantees that, for any node, the sum of the weight on the edges leaving that node is equal to 1.

Algorithm 2. Probabilistic top-down enumerator for pCFG-synth

```
1: procedure ENUMERATE(W_G, φ, cex )
2:     prog ← W_G.S
3:     d ← 0
4:     previousProgs ← ∅
5:     P_G ← BUILDPCFG(W_G)
6:     while 1 do
7:         if prog ∈ Σ* then
8:             previousProgs ← previousProgs ∪ prog
9:             if ∀x⃗ ∈ cex. φ(prog, x⃗) then
10:                return prog
11:            else
12:                prog ← S
13:                d ← 0
14:        prog ← REPLACENONTERMINALS(prog, P_G)
15:        d ← d + 1
16:        if d = maxDepth then
17:            prog ← COMPLETEPROGRAM(prog, P_G)
18:            if prog ∈ PreviousPrograms then
19:                prog ← S
20:                d ← 0
21: procedure REPLACENONTERMINALS(prog, P_G)
22:    NT ← list of nonterminals in prog
23:    for α ∈ NT do
24:        (α × β) ~ Cat(|π[α]|, {ℙ[π[α]_1], ℙ[π[α]_2], ...})   ▷ Sample from distribution
25:        prog ← prog.{α → β}                                  ▷ apply rule to prog
26:    return prog
27: procedure COMPLETEPROGRAM(prog, P_G)▷ Replaces non-terminal symbols with
    terminal symbols
28:    NT ← list of nonterminal symbols in prog
29:    for α ∈ NT do
30:        (α × β) ~ Cat(|π[α]|, {ℙ_Σ[π[α]_1], ℙ_Σ[π[α]_2], ...})   ▷ Sample
31:        prog ← prog.{nt → nt'}                               ▷ apply rule to prog
32:    return prog
```

We search this grammar tree using a top-down enumerative synthesizer, shown in Algorithm 2. This enumerates possible programs in the grammar in a top-down manner, expanding non-terminals by randomly sampling from the categorical distribution over the production rules. That is, the search algorithm starts by considering the node corresponding to the start symbol S. It then chooses the next node by sampling from a categorical distribution with event probabilities corresponding to the probabilities on the outgoing edges of the current node. The categorical distribution is a generalization of the Bernoulli distribution and describes the possible results of a random variable that can

take one of K possible categories, with the probability of each category separately specified. Formally, to sample a rule $\alpha \times \beta$ to apply to a non-terminal symbol α, we sample from the distribution:

$$(\alpha \times \beta) \sim Cat(|\pi[\alpha]|, \{\mathbb{P}[\pi[\alpha]_1], \mathbb{P}[\pi[\alpha]_2], \ldots\}),$$

where $|\pi[\alpha]|$ is the number of rules that could be applied to α and $\pi[\alpha]_i$ is the i^{th} of those rules, and $\{\mathbb{P}[\pi[\alpha]_1], \mathbb{P}[\pi[\alpha]_2], \ldots\}$ is a vector of probabilities corresponding to those rules.

We then apply the sampled rule, and repeat the process. We use $prog.\{\alpha \to \beta\}$ to indicate the result of substituting the first occurrence of α in a partial program $prog$ with β.

With a naive implementation of this algorithm, the probability of our algorithm generating any sentence s is equal to $\prod_{r_i \in D_s} \mathbb{P}[r_i]$, where D_s is the leftmost derivation of s. However, this will result in the algorithm generating the same programs multiple times, so we modify this algorithm in two ways: First, if we enumerate a complete program that we have seen before, we discard it; Second, we give a maximum depth limit, and if we are approaching the maximum depth limit, we sample only from the outgoing edges that result in complete programs.

Algorithm 3. pCFG-synth

```
1: procedure pCFG-synth(prompts, φ, G)
2:     conv ← [ ]
3:     progs ← ∅
4:     while prompts ≠ ∅ do
5:         response ← LLM(prompts.pop(), conv)
6:         conv.append(response)
7:         currentProg ← ExtractProgram(response)
8:         if ∀x⃗ φ(currentProg, x⃗) then
9:             return currentProg
10:        else
11:            progs ← progs ∪ currentProg
12:    W ← WeightCounter(prog, G)
13:    W_G ← (G, W)
14:    prog ← CEGIS(W_G, φ)
15:    return prog
```

5.3 Weighted A^* Search

We implement a second variation of pCFG-synth using the A^* weighted search algorithm as the underlying enumerator. A^* is a search algorithm that chooses which paths to extend based on minimizing the cost of the path so far and an estimate of the cost required to extend the path to the goal, i.e., it expands nodes that minimizes $f(x) = c(x) + g(x)$, where $c(x)$ is the cost of the path to

x so far and $g(x)$ is the estimated cost of reaching a goal node from x. This technique was first used for guiding synthesis by Lee et al. [27], and we adapted the algorithm from their work.

To implement our A^* search, we extend the definition of the grammar tree to a weighted grammar tree. Given a pCFG $P_G = (V, \Sigma, R, S, \mathbb{P})$, a weighted grammar tree $\mathcal{T}(W_G)$ is a directed labeled graph as defined before, but each edge has a corresponding weight, given as follows:

$$\omega(\alpha \to \beta) = \begin{cases} -\log_2(\mathbb{P}[\alpha \to \beta]) & \text{if } \mathbb{P}[\alpha \to \beta] > 0, \\ \inf & \text{otherwise.} \end{cases}$$

We use the negative log of the probability to ensure that higher weighted edges correspond to those with very low probabilities.

Algorithm 4. A^* search for pCFG-synth

```
1: procedure ENUMERATE(P_G, φ, cex )
2:     Q = {0, S}                                  ▷ Priority queue of candidates
3:     while Q ≠ ∅ do
4:         (f, prog) ← Q.pop()                     ▷ Remove program with minimal f
5:         if ∀x⃗ ∈ cex. φ(prog, x⃗) then
6:             return prog
7:         for (nt ∈ prog) × nt' do
8:             if (nt × nt') ∈ P_G.R then          ▷ For all applicable rules
9:                 prog ← prog.{nt → nt'}          ▷ apply rule to prog
10:                Q ← Q ∪ (c(prog) + g(prog), prog)
```

The A^* algorithm, shown in Algorithm 4, relies on two key functions: first, the function $c(x)$, which computes the cost of the path so far, and second, the function $g(x)$ which estimates the cost to extend the path to a goal node. Assuming x is a sentential form in our language, $c(x)$ and $g(x)$ are given by:

$$c(x) = \sum_{r_i \in D_x} -\log_2\left(\mathbb{P}[r_i]\right), \quad g(x) = \begin{cases} 0 & \text{if } x \in \Sigma^*, \\ -\sum_{x_i \in V} \log_2 h(x_i) & \text{otherwise,} \end{cases}$$

where x_i indicates the i^{th} symbol in x, and h is the upper bound of the probabilities of expressions that can be derived from x_i, and is calculated as the fixed point of:

$$\forall \alpha \in V. h(\alpha) = \max_{\alpha \to \beta \in R} \left(\mathbb{P}[\alpha \to \beta] \times \prod_{\beta_i \in V} h(\beta_i) \right),$$

The function $g(x)$ can then be thought of as the product of the probability of each non-terminal symbol in x being converted into a terminal symbol.

Smoothing the Probability Distributions: Since the A^* algorithm will not enumerate any programs whose derivation uses a rule with zero probability, we smooth the weighted grammar as follows, with $\gamma = 0.4$: $w'[\alpha \to \beta] = 10 \times \left(\frac{w[\alpha \to \beta]+1}{10}\right)^{\gamma}$.

6 Enumerative Synthesis with an Integrated LLM (iLLM-synth)

The disadvantage of the method described in the preceding section is that the language model cannot benefit from any additional information that the enumerator learns during enumeration, as all prompting happens prior to starting the enumerative synthesis. In this section we describe how we integrate an LLM into an enumerative synthesis algorithm, allowing it to update a probability distribution over the search grammar and to augment the grammar with new production rules, as shown in Algorithm 5.

Algorithm 5. Syntactic feedback generator

1: **procedure** SyntacticFeedback($W_G, prog, cex$)
2: $prompt \leftarrow$ GeneratePrompt($prog, cex$)
3: $response \leftarrow$ LLM($prompt$)
4: $candidate \leftarrow$ ExtractProgram($response$)
5: $W_G.W \leftarrow W_G.W +$ WeightCounter($response$)
6: $W_G.R \leftarrow W_G.R \cup (W_G.S \times response)$
7: **return** W_G

6.1 Integrated Prompting

We construct a prompt that asks the LLM to provide helper functions to assist a student in writing SMT-lib code. We give the LLM the constraints from the target synthesis problem and the partially complete program at the point the enumerator calls the LLM. If the LLM fails to solve the problem with this prompt, we later add the most recently failed candidate solution and the counterexample it failed on. These prompts are shorter than the prompts in those used in Sect. 4 and, therefore, cheaper and faster to run. An example Prompt 4 is as follows:

```
You are teaching a student to write SMT-LIB. The student must write a
    function that satisfies the following constraints:
(constraint (>= (fn vr0 vr1 vr2) vr0))
(constraint (>= (fn vr0 vr1 vr2) vr1))
(constraint (>= (fn vr0 vr1 vr2) vr2))
(constraint (or (= vr0 (fn vr0 vr1 vr2)) (or (= vr1 (fn vr0 vr1 vr2))
    (= vr2 (fn vr0 vr1 vr2)))))
So far, the student has written this code:
(define-fun fn ((vr0 Int) (vr1 Int) (vr2 Int)) Int
    (ite ?? ?? ??))
Can you suggest some helper functions for the student to use to
    complete this code and replace the ??
You must print only the code and nothing else.
```

Prompt 4. Integrated prompt for Example 1.

Algorithm 6. Top-down enumerator for iLLM-synth

```
1: procedure ENUMERATE(W_G, φ, cex )
2:     prog ← W_G.S
3:     d ← 0; i ← 0
4:     P_G ← BUILDPCFG(W_G)
5:     while 1 do
6:         if prog ∈ Σ* then
7:             if ∀x⃗ ∈ cex. φ(prog, x⃗) then
8:                 return prog
9:             else
10:                prog ← S
11:                d ← 0
12:        if i%n = 0 then
13:            W_G ← SYNTACTICFEEDBACK(W_G, prog, cex)
14:            P_G ← BUILDPCFG(W_G)
15:        prog ← REPLACENONTERMINALS(prog, P_G)
16:        d ← d + 1
17:        if d = maxDepth then
18:            prog ← COMPLETEPROGRAM(prog, P_G)
19:            if prog ∈ PreviousPrograms then
20:                prog ← S
21:                d ← 0
22:        i ← i + 1
```

6.2 Updating the Weighted Grammar

We initialize our algorithm with a weight of 1 for each rule in the grammar. We use the LLM-generated helper functions to augment the grammar in the following way: first, any helper functions will be added directly as new production rules to replace non-terminals of the correct type in the grammar. That is, if the LLM proposes the defined function f, a set of rules of the form $V_i \times f$ are added to the grammar, for all non-terminal symbols V_i such that this rule results in syntactically correct expressions, i.e., V_i must be of the same type as the co-domain of f. This is sufficient to guarantee syntactically correct expressions because any functions proposed by the LLM that are otherwise not well-formed, e.g., they reference variables that are not defined, are discarded. Any new rules are given a weight equal to the average of all the current weights for rules relevant to that non-terminal. The response parser also updates the weights of all existing rules in the grammar, according to Eq. 1, calculated from the set of helper functions the LLM proposed.

6.3 Integrating Syntactic Feedback into Enumerative Search

We integrate the syntactic feedback generator into the probabilistic enumerator, shown in Algorithm 3, and into the A^* weighted search, as shown in Algorithm 7. Both search algorithms call the syntactic feedback generator every n^{th} iteration,

where n is a heuristic used to ensure the LLM is not called with the same partial program repeatedly and that the search algorithm has time to exploit the information obtained from the LLM. Note that, when the probabilistic grammar is updated, the h values must be re-calculated in the A^* search.

Algorithm 7. A^* search for iLLM-synth

```
1: procedure ENUMERATE(P_G, φ, cex )
2:     Q = {0, S}                                  ▷ Priority queue of candidates
3:     i ← 0
4:     while Q ≠ ∅ do
5:         (f, prog) ← Q.pop()                     ▷ Remove program with minimal f
6:         if prog ∈ Σ* then
7:             if ∀x⃗ ∈ cex. φ(prog, x⃗) then
8:                 return prog
9:         if i%n = 0 then
10:            W_G ← SYNTACTICFEEDBACK(W_G, prog, cex)
11:            P_G ← BUILDPCFG(W_G)
12:        for (nt ∈ prog) × nt' do
13:            if (nt × nt') ∈ P_G.R then          ▷ For all applicable rules
14:                prog ← prog.{nt → nt'}          ▷ apply rule to prog
15:                Q ← Q ∪ (c(prog) + g(prog), prog)
16:        i ← i + 1
```

7 Evaluation

We evaluate our approaches on benchmarks taken from the SyGuS competition [3], each with a grammar that corresponds to the full language of their respective theories. We evaluate across three SyGuS categories: Bit-Vector (BV), Linear Integer Arithmetic (LIA), and Invariants (INV). We evaluate both the LLM as a stand-alone synthesizer, the probabilistic enumerator and A^* implementations with a pre-trained pCFG and the enumerator with a pre-trained syntactic oracle. We utilize OpenAI's GPT-3.5-turbo-16k model to generate the prompts used for the pre-trained pCFG and the standalone LLM evaluation because this model supports longer prompts. We configure this with a temperature of 1.0, conversation-style messaging. We use GPT-3.5-turbo for iLLM-synth, which has shorter prompts. We use the 4.8.12 64-bit version of Z3 for verification and cvc5 version 1.1.0 as a baseline.

Evaluation of the Stand-Alone LLM: We prompt the LLM until it produces up to 6 complete synthesis attempts per benchmark, with the results reported in line 1 of Table 1. Any incomplete solutions are discarded (i.e., functions without a function body), although these are relatively rare, and we discard only 0.85% of programs we generate. In total, the LLM solves 49% of benchmarks, performing better in the invariant and LIA categories than the bit-vector category. On

average, for the benchmarks it can solve, it takes 4 attempts to produce a correct solution. The average time for the LLM to generate a program is approximately $5s$ using the OpenAI Python API. However, this is dependent on OpenAI, and we report these times only as estimates in Table 1. We allow the LLM only 6 attempts to solve the problem since, by the 6^{th} iteration, the number of new solutions the LLM finds has dropped to <2% (and it finds 0 new solutions for LIA).

Evaluation of pCFG-synth: We evaluate both variants of pCFG-synth (with the probabilistic enumerator, denoted e-pCFG-synth, and with A^*, denoted A^*-pCFG-synth) using the wCFG obtained from the LLM. As a baseline, we run the same algorithms assigning a weight of 1 to every rule in the grammar (referred to as "enumerator" and A^* respectively in the results). pCFG-synth increases the number of benchmarks the probabilistic enumerator can solve by 30%, but barely increases the number A^* can solve, although the exact sets of benchmarks which A^* and A^*-pCFG-synth solve do differ significantly. We hypothesize that this is because A^*, guided by the pCFG with equal weights for all rules, is very good at generating short solutions, and A^*-pCFG-synth is worse at short solutions but better at generating more complex solutions guided by the pCFG.

We also report the results obtained by the union of the LLM alone and pCFG-synth, i.e., if the LLM solves the benchmark, we do not deploy the enumerator. This is a more realistic representation of how such a technique would be used and demonstrates that the enumerator can overcome shortcomings of the LLM and vice versa. The union of the LLM and A^*-pCFG-synth substantially outperforms cvc5, solving 73 more benchmarks.

Evaluating iLLM-synth: We evaluate both variants of iLLM-synth, denoted e-iLLM-synth and A^*-iLLM-synth. We set the temperature for e-iLLM-synth to 1, but find that A^*-iLLM-synth performs better with a temperature set to 0 which we hypothesize is due to the determinism of the algorithm. We find that iLLM-synth outperforms the enumerator of pCFG-synth, and gets close to the performance of cvc5, suggesting that the ability to prompt the LLM with additional information obtained during enumeration allows the LLM to provide better guidance to the enumerator, as well as to more frequently propose useful helper functions. We do find that iLLM-synth performs less well than methods incorporating the stand-alone LLM on the invariant benchmarks, which is likely because the invariant benchmarks benefit from the custom prompting technique described in Sect. 4.1. Future work would involve identifying further categories of benchmarks that benefit from custom prompts. It is worth noting that neither the probabilistic enumerator nor the A^* implementation includes many of the optimizations that mature solvers such as cvc5 implement, and yet, by integrating these simple algorithms with syntactic feedback from an LLM, they have achieved performance on par with the state-of-the-art enumerative solver.

Failure Modes: We manually examine a sample of the stand-alone LLM errors and give examples of such errors in the extended version of this paper[2]. Broadly, we identify the following common failures: Misunderstandings due to complex constraints (the LLM suggests solutions that are not syntactically close to the correct solution); simple syntactic errors, e.g., applying non-commutative operators to operands in the wrong order, concatenating bit-vectors in the wrong order or hallucinating operations; simple semantic errors, e.g., operators in the wrong order. Errors in the first category are not helpful to our guided enumerators, but the remaining categories of error still allow us to generate a wCFG that is likely to indicate the area of the solution. The benchmarks that cvc5 can solve and our enumerative techniques cannot, tend to have complex constraints and relatively short solutions that use less common operators (e.g., bitwise operators). We hypothesize that the LLM guidance becomes an impediment to the enumerator in these scenarios. In contrast, the average length (in characters) of a solution for benchmarks uniquely solved by the LLM is 4.7x the length of a solution for benchmarks uniquely solved by cvc5. Using the LLM to guide the enumerators increases the length of solutions that the enumerators can find, for instance all solutions found by A^* contain fewer than 3 operators, but A^*-iLLM-synth finds solutions with greater than 20 operators.

Programming-by-Example: We omit benchmarks from the syntax-guided synthesis competition tracks that solely focus on programming-by-example (PBE) (i.e., specifying a program only using input-output examples and a grammar). We omit these benchmarks for two reasons: first, since training data is trivial to generate for PBE, unlike general logical specifications [34], there are many other successful machine-learning driven synthesis techniques that can be trained for PBE techniques [6]. Second; our approaches are effective when the LLM can provide guidance to the enumerator, which comes from prompting the LLM with the logical constraints that form the specification. If we prompt the LLM using the prompting techniques outlined in Sect. 4.1 with a PBE specification, it tends to provide a solution in the form of a large case split over the input examples, which returns specific outputs for each input. This is not useful for guiding the enumerator because the LLM overfits to the examples in the specification and fails to provide any bias towards operators other than "if-then-else". To extend our approach to PBE, we would need to use a prompting approach tailored to input-output examples.

[2] https://arxiv.org/html/2403.03997.

Table 1. Summary of results. We run nondeterministic results, marked$^{\diamond}$, 3 times and report the average (standard deviation is less than 1% for all methods except the baseline enumerator for number of benchmarks solved). We highlight the best result in terms of number of benchmarks solved in each category. The timeout is 600 s. Times in *italic* indicate results that may vary depending on load on the OpenAI servers. The times for pCFG-synth do not include the time to call the standalone LLM and generate the wCFGs, but these are included in the times for LLM ∪ pCFG-synth.

	BV (384)		LIA (87)		INV (138)		Total (609)	
Methods	#	time(s)	#	time(s)	#	time(s)	#	%
LLM only	137	*13.5*	54	*7.10*	112	*29.2*	303	49.8%
e-pCFG-synth$^{\diamond}$	196.0	48.3	24.0	40.0	25.4	100.5	245.4	40.3%
A^*-pCFG-synth	262	60.1	35	72.7	25	99.7	322	52.9%
LLM ∪ e-pCFG-synth	255.0	*37.0*	64.0	*17.20*	117.7	*40.4*	436.7	71.7%
LLM ∪ A^*-pCFG-synth	**305.0**	*35.0*	65.0	*18.1*	**118.0**	*33.6*	**488.0**	80.1%
e-iLLM-synth$^{\diamond}$	241.0	*88.2*	63.4	*9.3*	65.3	*25.4*	370.0	60.8%
A^*-iLLM-synth$^{\diamond}$	272.3	*24.6*	**68.3**	*20.8*	67.3	*43.6*	408.0	67.0%
enumerator$^{\diamond}$	142.7	7.2	25.0	1.53	21.0	3.2	188.7	31.0%
A^*	253.0	25.4	34.0	73.19	22.0	31.1	309.0	50.7%
cvc5	292.0	17.1	43.0	19.53	80.0	23.6	415.0	68.1%

8 Threats to Validity

LLM Training Data: The SyGuS problems are publicly available and might be part of the training data for the LLM we use, although we believe the solutions were not publicly available at the time of training.

Reproducibility: These experiments use GPT-3.5, an LLM available via API from OpenAI. We have recorded the responses and parameters generated by the LLM in all experiments, but these may not be reproducible [33] since GPT-3.5 behaves non-deterministically in a way that cannot be seeded. However, we observe very small variations in the number of benchmarks solved in our experiments (although greater variation in the average solving time). It is also possible that OpenAI deprecates this LLM and its associated API or updates it and changes its behavior in the future.

Benchmark Bias: The benchmark set is taken from the SyGuS competition [3], but may not be very diverse and may not be representative of synthesis problems "in the wild". Nevertheless, this is a standard benchmark set used in many formal synthesis papers.

Hyperparameters: We have not invested time in parameter tuning, and better or worse results may be obtained by changing the LLM parameters (temperature), or adjusting the weights, enumeration depth and heuristic functions in the probabilistic enumerator and A^* algorithms.

9 Related Work

Many state-of-the-art of SyGuS solvers are based on enumerative synthesis [4,21, 27,37] and use clever heuristics to improve the search speed. Closest to our work is Euphony [27], which uses a pre-trained probabilistic higher-order grammar [9] to guide an A^* search. This requires a library of known solutions for training; an advantage of our approach is it exploits the availability of LLMs pre-trained on large bodies of code in other languages, and disregards the need for a library of known solutions of SyGuS problems for training. Weighted grammars have also been used to guide programming by example [30], and to encode syntactic objectives [20], for instance, for optimizing the length of solutions.

Almost all synthesis algorithms use oracles to give feedback to the synthesis process [24,25]. The majority of these use *semantic* oracles, which give feedback on the meaning of the program, for example, counterexamples [2]. The LLM in iLLM-synth can be considered a syntactic oracle as it only gives feedback on the syntax of the program. Two approaches [1,17] can be thought of as using syntactic oracles, which evaluate partial programs (or sentential forms) and tell the synthesizer whether a solution can be derived from the sentential form.

Machine learning techniques have been deployed to improve the efficiency of enumerative synthesis, e.g., reinforcement learning [12,15,34] or using neural networks to filter grammars for programming-by-examples problems [31].

LLMs, such as GPT-4 [32] and CoPilot [18], have demonstrated impressive capabilities in generating code and assisting in diverse programming tasks with natural language and input-output specifications [5,10,11,22]. However, their tendency to produce hallucinations, factually incorrect or contextually inappropriate outputs, which poses challenges to users [35,36,38]. Closest to our work is Kamath et al., who use LLMs to synthesize loop invariants [26] directly. Our work also demonstrates that LLMs are surprisingly good at synthesizing invariants, but also addresses the question of how to use LLMs in other formal synthesis problems and when they cannot find the solution in one shot. Other work that integrates formal methods with LLMs uses LLMs to generate program annotations for program annotation [41,43]. Jha et al. [23] and Song et al. [40] integrate an LLM into a CEGIS loop, but, unlike our work, the entire synthesis phase is implemented by an LLM, which does not allow them to benefit from the combined strengths of enumerative solving and LLMs.

10 Conclusions

We have presented a novel integration of LLMs into two enumerative synthesis algorithms, evaluated on benchmarks from the Syntax-Guided Synthesis competition. We found that LLMs and enumerative solvers have distinct strengths and weaknesses when deployed alone. We have demonstrated that, by allowing the enumerative synthesizer to prompt the LLM with information obtained during the enumeration and allowing the LLM to provide syntactic feedback to the enumeration, we can achieve performance that equals and exceeds the

state-of-the-art solvers, even with relatively simple enumerative algorithms. We argue that our results show that LLMs have the potential to make significant contributions in the domain of formal program synthesis, but the way to achieve this is by combining these techniques with existing algorithms in the literature. Enumerative synthesis is not dead yet!

Acknowledgements. This work was in part supported by an Amazon Research Award, a Royal Academy of Engineering research fellowship, and the European Research Council (ERC) project FormalWeb3 (Grant ID 101156734).

References

1. Abate, A., David, C., Kesseli, P., Kroening, D., Polgreen, E.: Counterexample guided inductive synthesis modulo theories. In: Chockler, H., Weissenbacher, G. (eds.) CAV 2018. LNCS, vol. 10981, pp. 270–288. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-96145-3_15
2. Alur, R., et al.: Syntax-guided synthesis. IEEE (2013)
3. Alur, R., Fisman, D., Singh, R., Udupa, A.: Syntax guided synthesis competition. https://sygus-org.github.io. Accessed 16 Jan 2024
4. Alur, R., Radhakrishna, A., Udupa, A.: Scaling enumerative program synthesis via divide and conquer. In: Legay, A., Margaria, T. (eds.) TACAS 2017. LNCS, vol. 10205, pp. 319–336. Springer, Heidelberg (2017). https://doi.org/10.1007/978-3-662-54577-5_18
5. Austin, J., et al.: Program synthesis with large language models. arXiv preprint arXiv:2108.07732 (2021)
6. Balog, M., Gaunt, A.L., Brockschmidt, M., Nowozin, S., Tarlow, D.: DeepCoder: learning to write programs. In: ICLR (Poster). OpenReview.net (2017)
7. Barbosa, H., et al.: cvc5: a versatile and industrial-strength SMT solver. In: TACAS 2022. LNCS, vol. 13243, pp. 415–442. Springer, Cham (2022). https://doi.org/10.1007/978-3-030-99524-9_24
8. Barrett, C.W., Sebastiani, R., Seshia, S.A., Tinelli, C.: Satisfiability modulo theories. In: Biere, A., Heule, M., van Maaren, H., Walsh, T. (eds.) Handbook of Satisfiability - Second Edition, Frontiers in Artificial Intelligence and Applications, vol. 336, pp. 1267–1329. IOS Press (2021). https://doi.org/10.3233/FAIA201017
9. Bielik, P., Raychev, V., Vechev, M.T.: PHOG: probabilistic model for code. In: ICML. JMLR Workshop and Conference Proceedings, vol. 48, pp. 2933–2942. JMLR.org (2016)
10. Brown, T., et al.: Language models are few-shot learners. In: Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901 (2020)
11. Bubeck, S., et al.: Sparks of artificial general intelligence: early experiments with GPT-4. arXiv preprint arXiv:2303.12712 (2023)
12. Bunel, R., Hausknecht, M., Devlin, J., Singh, R., Kohli, P.: Leveraging grammar and reinforcement learning for neural program synthesis. In: International Conference on Learning Representations (2018)
13. Chasins, S.E., Newcomb, J.L.: Using SyGuS to synthesize reactive motion plans. In: SYNT@CAV. EPTCS, vol. 229, pp. 3–20 (2016)
14. Chen, M., et al.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)

15. Chen, Y., Wang, C., Bastani, O., Dillig, I., Feng, Yu.: Program synthesis using deduction-guided reinforcement learning. In: Lahiri, S.K., Wang, C. (eds.) CAV 2020. LNCS, vol. 12225, pp. 587–610. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-53291-8_30
16. David, C., Kroening, D., Lewis, M.: Using program synthesis for program analysis. In: Davis, M., Fehnker, A., McIver, A., Voronkov, A. (eds.) LPAR 2015. LNCS, vol. 9450, pp. 483–498. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-48899-7_34
17. Feng, Y., Martins, R., Bastani, O., Dillig, I.: Program synthesis using conflict-driven learning. In: PLDI, pp. 420–435. ACM (2018)
18. GitHub, OpenAI: GitHub Copilot (2021)
19. Hart, P.E., Nilsson, N.J., Raphael, B.: A formal basis for the heuristic determination of minimum cost paths. IEEE Trans. Syst. Sci. Cybern. **4**(2), 100–107 (1968)
20. Hu, Q., D'Antoni, L.: Syntax-guided synthesis with quantitative syntactic objectives. In: Chockler, H., Weissenbacher, G. (eds.) CAV 2018. LNCS, vol. 10981, pp. 386–403. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-96145-3_21
21. Huang, K., Qiu, X., Shen, P., Wang, Y.: Reconciling enumerative and deductive program synthesis. In: PLDI, pp. 1159–1174. ACM (2020)
22. Jain, N., et al.: Jigsaw: large language models meet program synthesis. In: ICSE, pp. 1219–1231. ACM (2022)
23. Jha, S.K., et al.: Counterexample guided inductive synthesis using large language models and satisfiability solving. In: MILCOM 2023-2023 IEEE Military Communications Conference (MILCOM), pp. 944–949. IEEE (2023)
24. Jha, S., Gulwani, S., Seshia, S.A., Tiwari, A.: Oracle-guided component-based program synthesis. In: Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering-Volume 1, pp. 215–224 (2010)
25. Jha, S., Seshia, S.A.: A theory of formal synthesis via inductive learning. Acta Informatica **54**, 693–726 (2017)
26. Kamath, A., et al.: Finding inductive loop invariants using large language models (2023)
27. Lee, W., Heo, K., Alur, R., Naik, M.: Accelerating search-based program synthesis using learned probabilistic models. In: PLDI, pp. 436–449. ACM (2018)
28. Li, C., et al.: Large language models understand and can be enhanced by emotional stimuli. arXiv preprint arXiv:2307.11760 (2023)
29. Liang, P., Jordan, M.I., Klein, D.: Learning programs: a hierarchical Bayesian approach. In: ICML, pp. 639–646. Citeseer (2010)
30. Menon, A., Tamuz, O., Gulwani, S., Lampson, B., Kalai, A.: A machine learning framework for programming by example. In: International Conference on Machine Learning, pp. 187–195. PMLR (2013)
31. Morton, K., Hallahan, W.T., Shum, E., Piskac, R., Santolucito, M.: Grammar filtering for syntax-guided synthesis. In: AAAI, pp. 1611–1618. AAAI Press (2020)
32. OpenAI: GPT-4 technical report. arXiv, pp. 2303–08774 (2023)
33. Ouyang, S., Zhang, J.M., Harman, M., Wang, M.: LLM is like a box of chocolates: the non-determinism of ChatGPT in code generation. arXiv preprint arXiv:2308.02828 (2023)
34. Parsert, J., Polgreen, E.: Reinforcement learning and data-generation for syntax-guided synthesis. In: AAAI, pp. 10670–10678. AAAI Press (2024)
35. Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., Karri, R.: Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. In: 2022 IEEE Symposium on Security and Privacy (SP), pp. 754–768. IEEE (2022)

36. Perry, N., Srivastava, M., Kumar, D., Boneh, D.: Do users write more insecure code with AI assistants? In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, pp. 2785–2799 (2023)
37. Reynolds, A., Barbosa, H., Nötzli, A., Barrett, C., Tinelli, C.: CVC4SY: smart and fast term enumeration for syntax-guided synthesis. In: Dillig, I., Tasiran, S. (eds.) CAV 2019. LNCS, vol. 11562, pp. 74–83. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-25543-5_5
38. Sandoval, G., Pearce, H., Nys, T., Karri, R., Garg, S., Dolan-Gavitt, B.: Lost at C: a user study on the security implications of large language model code assistants. In: 32nd USENIX Security Symposium (USENIX Security 2023), pp. 2205–2222. USENIX Association, Anaheim, CA, August 2023
39. Solar-Lezama, A., Tancau, L., Bodik, R., Seshia, S., Saraswat, V.: Combinatorial sketching for finite programs. In: Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 404–415 (2006)
40. Song, C.H., Wu, J., Washington, C., Sadler, B.M., Chao, W.L., Su, Y.: LLM-planner: few-shot grounded planning for embodied agents with large language models. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 2998–3009 (2023)
41. Sun, C., Sheng, Y., Padon, O., Barrett, C.: Clover: closed-loop verifiable code generation. arXiv preprint arXiv:2310.17807 (2023)
42. Wei, J., et al.: Chain-of-thought prompting elicits reasoning in large language models. In: NeurIPS (2022)
43. Wu, H., Barrett, C., Narodytska, N.: Lemur: integrating large language models in automated program verification. arXiv preprint arXiv:2310.04870 (2023)